Chuleta examen: GitHub Actions

Solo Actions: teoría básica, sintaxis, ejercicios del profesor resueltos, secrets, Docker Hub y un ejemplo desde cero.

1. Conceptos básicos

Un **workflow** es un archivo YAML dentro de `.github/workflows/` (ruta exacta, con el punto delante; extensión `.yml` o `.yaml`). GitHub lo ejecuta cuando ocurre un evento (por ejemplo, un push). Los resultados se ven en la pestaña **Actions** del repo.

Estructura: **workflow → jobs → steps**.

- Cada job corre en una máquina nueva y vacía que se destruye al terminar.
- Los jobs corren **en paralelo**, salvo que uses `needs`.
- Los steps de un job se ejecutan en orden.
- El YAML usa **espacios** (2 por nivel), nunca tabuladores.

```
name: Mi workflow          # nombre (opcional)

on: [push]                 # cuándo se ejecuta

jobs:
  mi-job:                  # nombre del job
    runs-on: ubuntu-latest # en qué máquina
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hola"
```

2. Triggers (`on`)

```
on: [push]                       # cualquier push

on:
  push:
    branches: [main]             # solo en la rama main

on:
  push:
    branches: [test, development]

on:
  push:
```

```
    tags: ['v*']                # solo al subir un tag que empiece por v

on:
  pull_request:
    branches: [main]

on:
  push:
    branches: [main]
  workflow_dispatch:            # botón manual "Run workflow"
```

3. Runners (`runs-on`)

Sistema	Valor
Linux	`ubuntu-latest`
macOS	`macos-latest`
Windows	`windows-latest`

4. Steps: `uses`, `run`, `with`, `name`

- `uses`: usa una action ya hecha.
- `run`: ejecuta un comando de terminal.
- `with`: parámetros de la action.
- `name`: nombre del step (opcional, ayuda a leer los logs).

```
steps:
  - name: Descargar el código
    uses: actions/checkout@v4

  - name: Preparar Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.12"

  - name: Instalar dependencias
    run: pip install -r requirements.txt

  - name: Ejecutar script
    run: python euro.py USD

  - name: Varios comandos
    run: |
      echo "primero"
      echo "segundo"
```

Las dos actions clave:

- `actions/checkout@v4`: descarga el código del repo en el runner. Sin esto el runner está vacío y no encuentra tus archivos.
- `actions/setup-python@v5`: instala la versión de Python indicada en `python-version`.

Orden típico: **checkout → setup-python → pip install → ejecutar el script**.

5. Teoría de lo que aparece en los `run`

No hace falta saber programar. Los `run` son los mismos comandos que escribirías en una terminal.

- **paquete**: código que otra persona publicó para reutilizarlo (por ejemplo, `requests`, que sirve para hacer peticiones web).
- **`pip`**: el instalador de paquetes de Python. `pip install requests` instala `requests`.
- **¿Por qué en la action?** El runner es una máquina nueva y vacía en cada ejecución. Tiene Python (por `setup-python`) pero no tus librerías. Si el script hace `import requests` y no lo instalaste, falla con `ModuleNotFoundError`.
- **`requirements.txt`**: una lista de la compra, un paquete por línea. `pip install -r requirements.txt` instala todo lo de la lista (`-r` = read).

- **PyInstaller**: paquete que convierte un `.py` en un único ejecutable que funciona sin Python. `pyinstaller --onefile script.py` deja el resultado en `dist/`. En Windows el archivo acaba en `.exe`. Se hace en 3 sistemas porque un ejecutable de Windows no sirve en Linux.

Comando	Qué hace
`python euro.py USD`	Ejecuta el script; `USD` es un argumento
`python --version`	Muestra la versión de Python
`pip install -r requirements.txt`	Instala los paquetes de la lista
`chmod +x archivo`	Da permiso de ejecución (Linux/macOS)
`./archivo`	Ejecuta un archivo de la carpeta actual
`echo "hola"`	Imprime texto en el log

6. Ejercicios del profesor resueltos

a) `requirements.txt` en vez de `pip install requests`

Archivo `requirements.txt`:

```
requests
```

Step:

```
- run: pip install -r requirements.txt
```

Hay que subir `requirements.txt` al repo (`git add`, `commit`, `push`).

b) Rama `test` con Python 3.12

```
git checkout -b test
# edito main.yml: python-version: "3.12"
git add .
git commit -m "python 3.12 en test"
git push -u origin test
```

Cada rama tiene su propia copia del workflow, así que el cambio solo afecta a `test`.

c) Rama `development` ejecutando solo USD

```
git checkout main
git checkout -b development
# edito main.yml: run: python euro.py USD
git add .
git commit -m "solo USD en development"
git push -u origin development
```

Alternativa en un solo archivo, con condición por rama:

```
- run: python euro.py USD
  if: github.ref == 'refs/heads/development'
```

d) Tres jobs con PyInstaller

```
name: Build executables
on: [push]

jobs:
  build-linux:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install pyinstaller
      - run: pyinstaller --onefile comprova_desplegaments.py
      - uses: actions/upload-artifact@v4
        with:
          name: comprova-linux
          path: dist/comprova_desplegaments

  # build-macos (runs-on: macos-latest) y build-windows (runs-on: windows-
latest):
  # igual, cambiando runs-on. En Windows el path es
dist/comprova_desplegaments.exe

  test-executable:
    needs: build-linux
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
```

```
            name: comprova-linux
        - run: |
            chmod +x comprova_desplegaments
            ./comprova_desplegaments
```

7. Artifacts y `needs`

Cada job corre en una máquina distinta que se destruye al acabar, así que un archivo creado en un job desaparece.

- `upload-artifact`: guarda el archivo en GitHub antes de que se destruya la máquina.
- `download-artifact`: lo descarga en otro job.
- `needs: nombre-job`: espera a que ese job termine bien. Sin `needs`, los jobs van en paralelo.

8. Secrets

Problema: para subir una imagen a Docker Hub el workflow necesita usuario y contraseña. Si los escribes en el YAML, cualquiera que vea el repo los ve.

Solución: los Secrets se guardan cifrados en GitHub, fuera del código. En el workflow solo pones el nombre y GitHub sustituye el valor real al ejecutar. En los logs aparece como `***`.

Crear un secret:

1. Repo → **Settings** → **Secrets and variables** → **Actions**.
2. Pestaña **Secrets** → **New repository secret**.
3. **Name** (por ejemplo `DOCKERHUB_USERNAME`) y **Secret** (el valor real) → **Add secret**.
4. Repetir con `DOCKERHUB_TOKEN`.

Uso:

```
username: ${{ secrets.DOCKERHUB_USERNAME }}
password: ${{ secrets.DOCKERHUB_TOKEN }}
```

`${{ ... }}` significa "sustituye por el valor". El nombre debe coincidir exactamente con el que creaste.

Token de Docker Hub: foto de perfil → **Account settings** → **Security** (o **Personal access tokens**) → **New Access Token** (permiso Read & Write). Cópialo al crearlo, luego no se vuelve a mostrar. Úsalo en vez de tu contraseña.

Variables de entorno normales (no secretas):

```
env:
  MONEDA: USD
steps:
  - run: python euro.py $MONEDA
```

9. Subir una imagen a Docker Hub desde una action

Antes: crear los secrets `DOCKERHUB_USERNAME` y `DOCKERHUB_TOKEN` y tener un `Dockerfile` en la raíz del repo.

```
name: Docker
on:
  push:
    branches: [main]

jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/euro:latest
```

Versión solo con comandos (también válida):

```
      - run: |
          echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login -u "${{
secrets.DOCKERHUB_USERNAME }}" --password-stdin
          docker build -t ${{ secrets.DOCKERHUB_USERNAME }}/euro:latest .
          docker push ${{ secrets.DOCKERHUB_USERNAME }}/euro:latest
```

Comprobación: en hub.docker.com → **Repositories** debe aparecer `tu_usuario/euro`.

10. Matrix (opcional)

Ejecuta el mismo job con varias combinaciones sin repetirlo:

```
jobs:
  build:
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest, windows-latest]
        python: ["3.11", "3.12"]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
      - run: python --version
```

11. Ejemplo desde cero

Cómo queda todo al final

```
actions/                    <- carpeta del repo (la raíz)
├── .git/                   <- la crea git, no la toques
├── .github/
│   └── workflows/
│       └── main.yml        <- el workflow
├── .gitignore
├── euro.py                 <- el script
└── requirements.txt        <- lista de paquetes
```

Lo único que no puedes cambiar es la ruta `.github/workflows/`. Si la escribes mal, GitHub ignora el archivo y no pasa nada.

Paso 1: crear el repo en GitHub

1. Arriba a la derecha pulsa **+** → **New repository**.
2. **Repository name:** `actions`. Elige **Public**.
3. Marca **Add a README file** y pulsa **Create repository**.

Paso 2: clonarlo

En el repo, botón verde **Code** → copia la URL HTTPS:

```
git clone https://github.com/TU_USUARIO/actions.git
cd actions
```

Paso 3: crear los archivos

`euro.py` (versión simple para probar):

```
import sys
import requests

moneda = sys.argv[1] if len(sys.argv) > 1 else "TODAS"
print(f"Moneda pedida: {moneda}")
print(f"requests instalado, versión {requests.__version__}")
```

`requirements.txt`:

```
requests
```

Crear carpetas (en PowerShell: `mkdir .github\workflows`):

```
mkdir -p .github/workflows
```

`.github/workflows/main.yml`:

```
name: Probar mi script

on:
  push:
    branches: [main]

jobs:
  ejecutar:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar el código
        uses: actions/checkout@v4

      - name: Preparar Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Instalar dependencias
```

```
        run: pip install -r requirements.txt

      - name: Ejecutar el script
        run: python euro.py USD
```

Paso 4: subirlo

```
git add .
git commit -m "añado script y workflow"
git push
```

Si pide contraseña: usa un **Personal Access Token** (foto de perfil → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, con permisos **repo** y **workflow**).

Paso 5: ver la action funcionando

1. Pestaña **Actions** del repo (junto a *Pull requests*).
2. A la izquierda el nombre del workflow; en el centro una ejecución por cada push.
3. Estado: círculo amarillo = en ejecución, check verde = bien, cruz roja = falló.
4. Pulsa en la ejecución y luego en el job (`ejecutar`).
5. Pulsa en un step para desplegar su salida. En el último debe salir `Moneda pedida: USD` .

Paso 6: ramas `test` y `development`

Cambia el trigger a `on: [push]` para que corra en todas las ramas, y súbelo. Después crea cada rama como en el apartado 6 (b y c). Para comprobarlo: pestaña **Actions**, cada ejecución muestra la rama que la lanzó (filtro **Branch**). En `test` mira el step de Python; en `development` , que ejecuta `python euro.py USD` .

Paso 7: probar un secret

1. **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
2. **Name:** `MI_SECRETO` , **Secret:** `hola123` → **Add secret**.
3. Añade el step `run: echo "El secreto es ${{ secrets.MI_SECRETO }}"` , haz commit y push.
4. En el log saldrá `El secreto es ***` .

Paso 8: lanzar a mano y repetir

Con `workflow_dispatch:` en el `on`: **Actions** → workflow a la izquierda → **Run workflow** → rama → **Run workflow**. Para repetir una ejecución sin cambiar nada: entra en ella → **Re-run all jobs**.

Ciclo de trabajo

1. Edito en local.
2. `git add .` → `git commit -m "..."` → `git push`.
3. **Actions** → entro en la ejecución → miro los steps.
4. Verde = listo. Rojo = leo el error, corrijo y vuelvo al paso 1.

12. Depurar un workflow

Pestaña **Actions** → clic en la ejecución → clic en el job → desplegar el step que falló.

Error	Causa
El workflow no se ejecuta	Archivo fuera de `.github/workflows/`, mala extensión o no hiciste push
`No such file or directory`	Falta `actions/checkout@v4` o el archivo no se subió
`ModuleNotFoundError: requests`	Falta el `pip install` o `requirements.txt` mal
`Could not open requirements file`	`requirements.txt` no está en el repo o estás en otra rama
Error de YAML	Indentación (espacios, no tabuladores)
Docker login falla	Secrets mal escritos o token sin permisos
El workflow de una rama no cambia	Lo editaste en otra rama
La pestaña Actions está vacía	No hiciste push o la ruta del workflow es incorrecta

13. Resumen de una página

```
# Ruta: .github/workflows/algo.yml
name: Nombre
on:
  push:
    branches: [main]      # o tags: ['v*'], o workflow_dispatch:
```

```
jobs:
  mi-job:
    runs-on: ubuntu-latest   # macos-latest / windows-latest
    needs: otro-job          # (opcional) esperar a otro job
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python euro.py USD
      - uses: actions/upload-artifact@v4     # guardar archivo
        with:
          name: nombre
          path: dist/archivo
# Secrets: ${{ secrets.NOMBRE }}  (Settings -> Secrets and variables ->
Actions)
```